

DOCUMENTAÇÃO TÉCNICA

Dungeon Game — Godot 4

Guia de Integração entre Sistemas

Versão 1.0

1. Visão Geral do Projeto

Este documento descreve a arquitetura de comunicação entre os sistemas do jogo. O objetivo é garantir que os programadores responsáveis por cada módulo (Player, Loja, Inimigos, Salas) consigam integrar seu trabalho sem depender diretamente do código dos outros.

⚠ Leia a Seção 2 (EventBus) antes de qualquer outra coisa. Ele é o coração de toda a comunicação do jogo.

1.1 Estrutura de Pastas

```
autoload/          → Scripts globais carregados automaticamente
  EventBus.gd      → Canal central de eventos (LEIA PRIMEIRO)
  GameManager.gd   → Estado global do jogo (fase, moedas)
  PlayerStats.gd   → Atributos do jogador (vida, dano, armadura)

inimigos/
  Base/            → Scripts herdados por todos os inimigos
  Boss/            → Chefe final
  GreaterDemon/    → Inimigo de elite
  MiniDemon/       → Inimigo básico
  Wizard/          → Inimigo à distância

salas/
  RoomManager.gd   → Lógica de spawn e contagem de mortes
  ClosedRoom.gd    → Reage ao EventBus para abrir/fechar portas

loja/              → Sistema de compras
player/            → Cenas e scripts do jogador
ui/                → HUD, GameOver, Créditos
fases/            → Cenas das fases (Fase1, Fase2, Fase3)
```

2. EventBus — Canal Central de Eventos

O EventBus é um Autoload (singleton) que permite comunicação entre qualquer parte do jogo sem referências diretas. Em vez de um script chamar diretamente outro, ele emite um sinal. Qualquer sistema interessado escuta esse sinal.

 Regra de ouro: nunca use `get_node()` para acessar sistemas de outros programadores. Use o EventBus.

2.1 Como funciona

Ação	Como fazer
Emitir um evento	<code>EventBus.emit_signal("nome_do_sinal", parametros)</code>
Escutar um evento	<code>EventBus.nome_do_sinal.connect(minha_funcao)</code>
Quando conectar	Sempre no <code>_ready()</code> do seu script

2.2 Sinais disponíveis

Sinais de Inimigos

```
# Emitido por: EnemyBase.gd (die())
# Escutado por: RoomManager.gd, LootSystem, Boss
signal inimigo_morto(inimigo: Node, posicao: Vector2)

# Emitido por: LootSystem (quando inimigo morre com loot)
# Escutado por: Player, HUD
signal item_dropado(item_id: String, posicao: Vector2)
```

Sinais de Sala

```
# Emitido por: RoomManager.gd (iniciar_combate())
# Escutado por: ClosedRoom.gd
signal sala_iniciada(sala_id: String)

# Emitido por: RoomManager.gd (finalizar_sala())
# Escutado por: ClosedRoom.gd
signal sala_limpa(sala_id: String)
```

Sinais do Player

```
# Emitido por: PlayerStats.gd (tomar_dano()) quando vida = 0
# Escutado por: GameManager, HUD, GameOver
signal player_morreu
```

```
# Emitido por: PlayerStats.gd (tomar_dano())
# Escutado por: HUD
signal player_tomou_dano(vida_atual: int, vida_maxima: int)

# Emitido por: PlayerStats.gd (set_vida())
# Escutado por: HUD
signal player_curou(vida_atual: int, vida_maxima: int)

# Emitido por: GameManager.gd ou PlayerStats.gd
# Escutado por: HUD, Loja
signal moedas_atualizadas(total: int)
```

3. Autoloads — Scripts Globais

3.1 GameManager.gd

Responsável pelo estado global da partida: fase atual, moedas e controle de game over.

Propriedade / Método	Descrição
moedas: int	Total de moedas do jogador
fase_atual: int	Índice da fase em andamento
jogo_ativo: bool	false quando game over
adicionar_moedas(qtd)	Soma moedas e emite moedas_alteradas
gastar_moedas(qtd) -> bool	Gasta se tiver saldo. Retorna true/false
mudar_fase(cena_path)	Troca de cena e emite fase_alterada

● **CORREÇÃO NECESSÁRIA:** GameManager tem seu próprio sinal moedas_alteradas, mas o EventBus também tem moedas_atualizadas. Use apenas o EventBus para evitar duplicidade. Atualize GameManager para emitir EventBus.emit_signal("moedas_atualizadas", moedas) ao invés do sinal local.

3.2 PlayerStats.gd

Fonte da verdade sobre o estado do jogador. Qualquer sistema que precise saber vida ou aplicar dano deve usar este autoload.

Propriedade / Método	Descrição
vida: int	Vida atual do jogador
vida_max: int	Vida máxima (padrão: 5)
armadura: int	Absorve dano antes da vida
dano: int	Dano base do jogador
velocidade_mov: float	Velocidade de movimento
itens: Array	Lista de itens adquiridos
set_vida(valor)	Define vida com clamp e emite sinal
tomar_dano(dano_bruto)	Desconta armadura, aplica dano, verifica morte

● **CORREÇÃO NECESSÁRIA:** tomar_dano() não emite vida_alterada nem player_morreu. Veja a Seção 6 para o código corrigido.

Como usar — Programador da Loja

```
# Adicionar item que aumenta vida máxima
func comprar_item_vida():
    if GameManager.gastar_moedas(10):
        PlayerStats.vida_max += 2
        PlayerStats.set_vida(PlayerStats.vida) # força atualização do HUD

# Adicionar armadura
func comprar_armadura():
    if GameManager.gastar_moedas(15):
        PlayerStats.armadura += 1
        EventBus.emit_signal("armadura_alterada", PlayerStats.armadura)
```

Como usar — Programador do Player

```
# No script do player, delegue dano para o autoload
func take_damage(amount: int):
    PlayerStats.tomar_dano(amount)

# Para curar
func curar(quantidade: int):
    PlayerStats.set_vida(PlayerStats.vida + quantidade)
```

4. Sistema de Inimigos

4.1 Hierarquia de herança

```
EnemyBase (CharacterBody2D)  ← Base de todos os inimigos
├── ChasePlayer              ← Inimigo terrestre com perseguição
│   └── MiniDemon            ← Herda ChasePlayer
├── EnemyRegent (Wizard)     ← Inimigo à distância com fuga
└── Boss                     ← Chefe (NÃO herda EnemyBase atualmente)
```

⚠ O Boss não herda EnemyBase. Isso significa que ele não usa `take_damage()` padronizado nem emite `inimigo_morto`. Veja Seção 6 para as correções necessárias.

4.2 EnemyBase — Contrato público

Todo inimigo expõe os seguintes métodos e propriedades que outros sistemas podem usar:

Método / Propriedade	Descrição
<code>take_damage(amount: int)</code>	Aplica dano, knockback e flash vermelho
<code>die()</code>	Toca animação, emite <code>inimigo_morto</code> , remove o nó
<code>speed: float</code>	Velocidade de movimento (export)
<code>max_health: int</code>	Vida máxima (export)
<code>knockback_force: float</code>	Força do knockback ao receber dano (export)

4.3 Adicionando um novo inimigo

Siga este padrão para criar qualquer inimigo novo:

```
# inimigos/MeuInimigo/MeuInimigo.gd
extends Enemy # herda EnemyBase automaticamente

@export var minha_propriedade = 10

func _ready():
    super._ready() # SEMPRE chame o super

# Sobrescreva apenas o que for diferente
func move_logic(delta):
    super.move_logic(delta) # opcional: chama o comportamento base
    # sua lógica aqui
```

```
# NÃO sobrescreva die() sem chamar super.die()
# O super.die() é quem emite o EventBus
```

4.4 Wizard / EnemyRegent — Comportamento especial

O Wizard foge quando o player se aproxima e atira em cone de 3 projéteis. Propriedades configuráveis no editor:

Propriedade (export)	Valor padrão
projectile_scene	Arraste Projectile.tscn aqui
min_distance	150px — distância para começar a fugir
shoot_range	300px — alcance de tiro
fire_rate	2.0s — tempo entre rajadas

5. Sistema de Salas

5.1 Fluxo de uma sala

```
1. Player entra na AreaAtivacao
  → RoomManager._on_area_ativacao_body_entered()
  → iniciar_combate()
  → EventBus emite: sala_iniciada(sala_id)
  → ClosedRoom escuta e fecha as portas
  → spawn_inimigos()

2. Inimigo morre
  → EnemyBase.die()
  → EventBus emite: inimigo_morto(inimigo, posicao)
  → RoomManager._on_inimigo_derrotado() conta +1

3. Todos os inimigos morrem
  → finalizar_sala()
  → EventBus emite: sala_limpa(sala_id)
  → ClosedRoom escuta e abre as portas
```

5.2 RoomManager — Configuração no editor

Propriedade (export)	Como configurar
sala_id	ID único por sala (ex: sala_01, sala_02)
inimigos_scenes	Arraste as cenas .tscn dos inimigos aqui
quantidade_para_vencer	Quantos inimigos precisam morrer para limpar

 Cada sala deve ter um sala_id único. Se duas salas tiverem o mesmo ID, ambas vão abrir quando qualquer uma for limpa.

5.3 ClosedRoom — Configuração no editor

O ClosedRoom.gd deve ter o mesmo sala_id configurado que o RoomManager da mesma sala. Ele filtra os eventos para reagir apenas à sua própria sala.

```
# ClosedRoom.gd - como conectar as portas
func _on_sala_iniciada(id: String):
    if id != sala_id: return # ignora outras salas
    $PortaEsquerda.fechar() # substitua pela sua lógica
    $PortaDireita.fechar()

func _on_sala_limpa(id: String):
```

```
if id != sala_id: return  
$PortaEsquerda.abrir()  
$PortaDireita.abrir()
```

6. Guia — Programador da Loja

 Esta seção é especialmente para quem está desenvolvendo o sistema de loja.

6.1 O que a loja pode fazer

- Verificar saldo de moedas via `GameManager.moedas`
- Gastar moedas via `GameManager.gastar_moedas(qtd)` — retorna `true/false`
- Modificar atributos do player via `PlayerStats`
- Escutar `EventBus.moedas_atualizadas` para atualizar a UI da loja

6.2 Atributos que a loja pode modificar

Atributo em PlayerStats	Efeito no jogo
vida_max	Aumenta a vida máxima do jogador
armadura	Absorve dano antes de reduzir vida
dano	Aumenta o dano dos ataques do player
velocidade_mov	Aumenta velocidade de deslocamento
velocidade_ataque	Reduz o cooldown entre ataques

6.3 Template de item da loja

```
# loja/Shop.gd
func _ready():
    # Escuta atualizações de moeda para atualizar UI
    EventBus.moedas_atualizadas.connect(_atualizar_ui_moedas)

func tentar_comprar_item(custo: int, tipo: String) -> bool:
    if not GameManager.gastar_moedas(custo):
        print("Moedas insuficientes!")
        return false

    match tipo:
        "vida":
            PlayerStats.vida_max += 2
            PlayerStats.set_vida(PlayerStats.vida_max) # cura completa
        "armadura":
            PlayerStats.armadura += 1
        "dano":
            PlayerStats.dano += 1
        "velocidade":
            PlayerStats.velocidade_mov += 20.0
```

```
    PlayerStats.itens.append(tipo)
    return true

func _atualizar_ui_moedas(total: int):
    $LabelMoedas.text = "Moedas: " + str(total)
```

7. Guia — Programador do Player

 Esta seção é especialmente para quem está desenvolvendo o personagem jogável.

7.1 Responsabilidade do script player.gd

O player.gd cuida apenas de input, animação e movimento. A gestão de vida e atributos fica no autoload PlayerStats.gd.

O player.gd faz	O PlayerStats.gd faz
Lê input do teclado/controla	Guarda vida, armadura, dano
Controla animações	Calcula e aplica dano recebido
Movimento e colisão	Emita sinais de vida para o HUD
Chama PlayerStats.tomar_dano()	Emita player_morreu quando vida = 0

7.2 Como receber dano corretamente

```
# player/player.gd
func take_damage(amount: int):
    PlayerStats.tomar_dano(amount)
    # NÃO manipule vida aqui - deixe o PlayerStats fazer isso

# Para verificar se pode atacar (usa dano do PlayerStats)
func atacar(alvo: Node):
    if alvo.has_method("take_damage"):
        alvo.take_damage(PlayerStats.dano)
```

7.3 Como o HUD deve escutar o player

```
# ui/HUD.gd
func _ready():
    EventBus.player_tomou_dano.connect(_on_vida_alterada)
    EventBus.player_curou.connect(_on_vida_alterada)
    EventBus.moedas_atualizadas.connect(_on_moedas_alteradas)
    EventBus.player_morreu.connect(_on_game_over)

func _on_vida_alterada(vida_atual: int, vida_maxima: int):
    $BarraVida.value = vida_atual
    $BarraVida.max_value = vida_maxima

func _on_moedas_alteradas(total: int):
```

```
$LabelMoedas.text = str(total)
```

```
func _on_game_over():  
    get_tree().change_scene_to_file("res://ui/GameOver.tscn")
```

8. Correções Necessárias

🔴 Os itens abaixo são bugs ou inconsistências encontrados na revisão do código atual. Cada programador deve aplicar a correção no seu módulo.

8.1 PlayerStats.gd — tomar_dano() incompleto

O método atual não emite sinal e não verifica morte. Substitua:

```
# ☒ PlayerStats.gd - tomar_dano() corrigido
func tomar_dano(dano_bruto: int):
    var dano_restante = dano_bruto
    if armadura > 0:
        var absorvido = min(armadura, dano_bruto)
        armadura -= absorvido
        dano_restante -= absorvido
        emit_signal("armadura_alterada", armadura)
    if dano_restante > 0:
        vida -= dano_restante
        vida = max(vida, 0)
        emit_signal("vida_alterada", vida) # ← ADICIONADO
        EventBus.emit_signal("player_tomou_dano", vida, vida_max) # ← ADICIONADO
    if vida <= 0:
        EventBus.emit_signal("player_morreu") # ← ADICIONADO
```

8.2 Boss.gd — minions e morte sem EventBus

```
# ☒ Boss.gd - _ready() corrigido
func _ready():
    # ...
    EventBus.inimigo_morto.connect(_on_minion_derrotado) # ← troca tree_exited

func _on_minion_derrotado(inimigo: Node, posicao: Vector2):
    minions_vivos -= 1

# ☒ Boss.gd - die() corrigido
func die():
    if is_dead: return
    is_dead = true
    # ... código existente ...
    await anim.animation_finished
    EventBus.emit_signal("inimigo_morto", self, global_position) # ← ADICIONADO
    queue_free()
```

8.3 EnemyBase.gd — ordem errada no die()

```
# ✗ Atual - emite antes da animação terminar
EventBus.emit_signal("inimigo_morto", self, global_position)
await anim.animation_finished
queue_free()

# ☑ Correto - emite depois da animação
await anim.animation_finished
EventBus.emit_signal("inimigo_morto", self, global_position)
queue_free()
```

8.4 ClosedRoom.gd — não filtra sala_id

```
# ☑ ClosedRoom.gd - adicione o export e os filtros
@export var sala_id: String = "sala_01"

func _on_sala_iniciada(id: String):
    if id != sala_id: return # ← ADICIONADO
    print("Sala fechada! Portas travadas.")

func _on_sala_limpa(id: String):
    if id != sala_id: return # ← ADICIONADO
    print("Sala limpa! Portas abertas.")
```

9. Fluxo Completo de Integração

Este é o caminho completo de um evento desde o início de uma sala até a loja:

```
Player entra na sala
→ RoomManager.iniciar_combate()
→ EventBus: sala_iniciada("sala_01")
    → ClosedRoom fecha as portas
→ spawn_inimigos()

Inimigo ataca o player
→ player.take_damage(1)
→ PlayerStats.tomar_dano(1)
→ EventBus: player_tomou_dano(vida_atual, vida_max)
    → HUD atualiza barra de vida

Player derrota inimigo
→ EnemyBase.die()
→ EventBus: inimigo_morto(inimigo, posicao)
    → RoomManager conta +1
    → LootSystem dropa moeda

Player coleta moeda
→ GameManager.adicionar_moedas(1)
→ EventBus: moedas_atualizadas(total)
    → HUD atualiza contador
    → Loja atualiza UI

Todos os inimigos mortos
→ RoomManager.finalizar_sala()
→ EventBus: sala_limpa("sala_01")
    → ClosedRoom abre as portas
```

Dúvidas? Consulte o responsável pela arquitetura antes de criar referências diretas entre módulos.